Task 5: Istio Service Mesh

Knowledge Assessment

1. Istio's Role in Kubernetes – The Sidecar Proxy Model

Istio is a service mesh that adds a dedicated infrastructure layer for service-to-service communication inside a Kubernetes cluster. It handles traffic management, security (mTLS), and observability without changing application code.

When Istio is installed and a namespace is labeled for injection (istio-injection=enabled), Istio automatically injects an Envoy sidecar proxy container into every Pod. This sidecar intercepts all inbound and outbound traffic using iptables rules, so the application container never talks directly to the network.

In this repo, the backend deployment already runs two containers ,the app and a manually managed jaeger-agent sidecar. With Istio, tracing, mTLS, and traffic control would be handled by the Envoy sidecar instead:

```
# manifests/backend-deployment.yaml (current)
spec:
  template:
    metadata:
      labels:
        app: backend          # <-- Istio uses this label for traffic rules
    spec:
      containers:
      - name: backend
        image: premkoredla/backend:a439c720410ac53ef1f63d0ed30108a8e0998be7
        ports:
        - containerPort: 5000
      - name: jaeger-agent    # <-- With Istio, this sidecar becomes redundant;
        image: jaegertracing/jaeger-agent:1.55   # Envoy handles tracing natively
```

To enable Istio injection for the default namespace:

```
kubectl label namespace default istio-injection=enabled
```

After labeling, every new Pod gets an auto-injected istio-proxy sidecar:

```
Pod: backend-xxxx
├── container: backend       (app, port 5000)
└── container: istio-proxy   (Envoy, intercepts all traffic)
```

This repo already exposes /metrics on the backend (scraped by Prometheus via ServiceMonitor). With Istio, the Envoy proxy adds additional L7 metrics (request rate, error rate, latency) automatically.

2. PeerAuthentication vs AuthorizationPolicy – Enforcing Strict mTLS

PeerAuthentication

Controls how traffic is received by a workload,.specifically whether mTLS is required for inbound connections. It operates at the transport layer.

AuthorizationPolicy

Controls who is allowed to talk to a workload, which source identities (SPIFFE/SVID) or request attributes (headers, methods, paths) are permitted. It operates at the L4/L7 authorization layer.

Enforcing Strict mTLS Across the default Namespace

This applies to all services in this repo (frontend, backend, mongodb):

```yaml
# istio/peer-authentication.yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default-strict-mtls
  namespace: default          # applies to all pods in the namespace
spec:
  mtls:
    mode: STRICT              # reject any plaintext traffic
```

AuthorizationPolicy – Only Frontend Can Call Backend

This mirrors the existing NetworkPolicy in manifests/network-policy.yaml but at the Istio identity layer:

```yaml
# istio/authz-backend.yaml
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: backend-allow-frontend
  namespace: default
spec:
  selector:
    matchLabels:
      app: backend            # target: backend pods
  rules:
  - from:
    - source:
        principals:
          # SPIFFE identity of the frontend service account
          - "cluster.local/ns/default/sa/frontend"
    to:
    - operation:
        ports: ["5000"]
```

Key Difference

PeerAuthentication - Transport security
AuthorizationPolicy - Access control (L4/L7)

3. Traffic Management – Canary Deployment with VirtualService & DestinationRule

Istio's traffic model separates routing logic (VirtualService) from backend definitions (DestinationRule). This enables fine-grained traffic splitting without changing Kubernetes Service or Deployment objects.

Deploy a Canary Version of Backend

Adding a version label to differentiate stable from canary:

```
# manifests/backend-deployment.yaml (stable - add version label)
spec:
  template:
    metadata:
      labels:
        app: backend
        version: stable
```

```
# manifests/backend-canary-deployment.yaml (new canary deployment)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-canary
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
      version: canary
  template:
    metadata:
      labels:
        app: backend
        version: canary
    spec:
      containers:
      - name: backend
        image: premkoredla/backend:<new-canary-sha>
        ports:
        - containerPort: 5000
```

DestinationRule (Define Subsets)

```
# istio/backend-destination-rule.yaml
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: backend
  namespace: default
spec:
```

```
  host: backend.default.svc.cluster.local
  subsets:
  - name: stable
    labels:
      version: stable
  - name: canary
    labels:
      version: canary
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL      # enforce mTLS between services
```

VirtualService (Split Traffic 90/10)

```
# istio/backend-virtual-service.yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: backend
  namespace: default
spec:
  hosts:
  - backend.default.svc.cluster.local
  http:
  - route:
    - destination:
        host: backend.default.svc.cluster.local
        subset: stable
      weight: 90
    - destination:
        host: backend.default.svc.cluster.local
        subset: canary
      weight: 10
```

Promote or Rollback

Once canary metrics (error rate, latency from Prometheus/Jaeger) look good, shift to 100% canary by updating the weight values, or rollback by setting canary weight back to 0.

4. Istio Ingress Gateway vs. Kubernetes Ingress Controller

Kubernetes Ingress (NGINX) primarily supports HTTP and HTTPS traffic, while Istio Ingress Gateway supports a broader range of protocols including HTTP, HTTPS, TCP, gRPC, and WebSocket. Traffic management in Kubernetes Ingress relies mainly on annotations with limited routing capabilities, whereas Istio uses Gateway and VirtualService resources that enable advanced features like retries, timeouts, and fault injection. Native mutual TLS (mTLS) is not supported inbound by standard NGINX Ingress, but Istio Ingress Gateway provides built-in mTLS enforcement through Envoy. Observability in Kubernetes Ingress is relatively limited, while Istio offers full Layer 7 metrics, distributed tracing, and detailed telemetry via Envoy proxies. Additionally, Kubernetes Ingress operates independently of a service mesh, whereas Istio Ingress Gateway is a dedicated entry point into the service mesh and participates fully in certificate rotation, policy enforcement, and telemetry management through Istiod.

Current Repo Setup (NGINX Ingress)

```yaml
# manifests/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: default
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  tls:
  - hosts:
    - myapp.local
    secretName: frontend-tls
  rules:
  - host: myapp.local
    http:
      paths:
      - path: /
        backend:
          service:
            name: frontend
            port:
              number: 80
      - path: /api
        backend:
          service:
            name: backend
            port:
              number: 5000
```

Istio Ingress Gateway Equivalent

An Istio Gateway replaces the Ingress resource and pairs with a VirtualService to route traffic:

```yaml
# istio/gateway.yaml
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: app-gateway
  namespace: default
spec:
  selector:
    istio: ingressgateway     # uses the Istio ingress gateway pod
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    tls:
      mode: SIMPLE
      credentialName: frontend-tls   # same secret used in current ingress.yaml
    hosts:
    - myapp.local

---
# istio/app-virtual-service.yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: app-routes
  namespace: default
spec:
  hosts:
  - myapp.local
  gateways:
  - app-gateway
  http:
  - match:
    - uri:
        prefix: /api
    route:
    - destination:
        host: backend.default.svc.cluster.local
        port:
          number: 5000
  - route:
    - destination:
        host: frontend.default.svc.cluster.local
        port:
          number: 80
```

5. Istio for Observability – Prometheus, Grafana, and Jaeger

Current Observability Stack in the Repo

The repo already deploys Prometheus, Grafana, and Jaeger manually across the observability namespace.

```
# observability/prometheus/prometheus-config.yaml
scrape_configs:
  - job_name: 'backend'
    static_configs:
      - targets: ['backend.default.svc.cluster.local:5000']
```

```
# observability/prometheus/service-monitor.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: backend-servicemonitor
  namespace: observability
spec:
  selector:
    matchLabels:
      app: backend
  endpoints:
  - port: http
    path: /metrics
    interval: 30s
```

```
# manifests/backend-deployment.yaml (manual jaeger-agent - replaced by Istio)
- name: jaeger-agent
  image: jaegertracing/jaeger-agent:1.55
  args:
    - "--reporter.grpc.host-port=jaeger.observability.svc.cluster.local:14250"
```

Istio Enhances the Stack

Prometheus

Istio's Envoy sidecars automatically expose metrics on port 15090. Add these scrape jobs to the existing prometheus-config.yaml:

```
scrape_configs:
  # Istio control plane metrics
  - job_name: 'istiod'
```

```
    kubernetes_sd_configs:
      - role: endpoints
        namespaces:
          names: ['istio-system']
    relabel_configs:
      - source_labels: [__meta_kubernetes_service_name]
        action: keep
        regex: istiod

  # Istio data plane (Envoy sidecars)  -auto-discovered
  - job_name: 'istio-mesh'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_container_name]
        action: keep
        regex: istio-proxy
```

Istio emits standard RED metrics automatically: istio_requests_total, istio_request_duration_milliseconds, istio_tcp_connections_opened_total.

Grafana

Istio provides pre-built Grafana dashboards. Import them via ConfigMap alongside the existing dashboard setup:

```
# observability/grafana/dashboards-configmap.yaml (adding Istio dashboards)
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-dashboards
  namespace: observability
data:
  # Dashboard IDs from grafana.com:
  # 7639 - Istio Mesh Dashboard
  # 7636 - Istio Service Dashboard (per-service latency/traffic/errors)
  # 7630 - Istio Workload Dashboard
```

Jaeger (Distributed Tracing)

Currently the backend runs a manual jaeger-agent sidecar. With Istio, Envoy automatically propagates trace headers (B3, W3C TraceContext) and sends spans to the existing Jaeger deployment - eliminating the need for the manual sidecar.

```
# istio/telemetry-tracing.yaml
apiVersion: telemetry.istio.io/v1alpha1
kind: Telemetry
metadata:
  name: tracing-config
  namespace: default
spec:
```

```
  tracing:
  - providers:
    - name: jaeger
    randomSamplingPercentage: 100   # 100% in dev; reduce in prod

# Patch to istio-system/istio ConfigMap (MeshConfig)
meshConfig:
  extensionProviders:
  - name: jaeger
    opentelemetry:
      service: jaeger.observability.svc.cluster.local
      port: 14268
```

With Istio, the manual jaeger-agent container in backend-deployment.yaml and the explicit Prometheus scrape job for backend.default.svc.cluster.local:5000 can both be removed the mesh handles them transparently.